

PROGRAMLAMA DİLLERİNİN EVRİMİ

Bölüm 2 — Evolution of the Major Programming Languages

Concepts of Programming Languages, 12. Baskı — Robert W. Sebesta

Kapsamlı Türkçe Konu Anlatımı, Sorular ve Çözümler

İçindekiler

1. Giriş ve Bölümün Amacı
2. Zuse'nin Plankalkül
3. Pseudocode Sistemleri (Short Code, Speedcoding, UNIVAC)
4. IBM 704 ve Fortran
5. Fonksiyonel Programlama: Lisp
6. İlk Sofistike Adım: ALGOL 60
7. İşletme Uygulamaları: COBOL
8. Zaman Paylaşımının Başlangıcı: Basic
9. Herkes İçin Her Şey: PL/I
10. Dinamik Diller: APL ve SNOBOL
11. Veri Soyutlamanın Başlangıcı: SIMULA 67
12. Ortogonal Tasarım: ALGOL 68
13. ALGOL Soyundan Gelenler: Pascal ve C
14. Mantık Programlama: Prolog
15. Tarihin En Büyük Tasarım Çabası: Ada
16. Nesne Yönelimli Programlama: Smalltalk
17. C++ ve Türevleri (Swift, Delphi)
18. Java
19. Betik Dilleri: Perl, JavaScript, PHP, Python, Ruby
20. C#
21. Biçimlendirme-Programlama Melezi: XSLT ve JSP
22. Olası Sınav Soruları ve Cevapları
23. Kitap Bölüm Sonu Soruları ve Çözümleri

1. Giriş ve Bölümün Amacı

Bu bölüm, programlama dillerinin tarihsel evrimini anlatır. Her dilin tasarlandığı ortamı, katkılarını ve sonraki dillere etkisini inceler. Amaç, çok sayıda dil özelliğinin nereden geldiğini anlayarak günümüz dillerini daha iyi kavramaktır.

Neden Dil Tarihi?

Dillerin evrimini anlamak, mevcut özelliklerin neden var olduğunu, hangi tasarım kararlarının başarılı ya da başarısız olduğunu kavramamızı sağlar. Her yeni dil, önceki dillerin eksikliklerini gidermeye çalışır.

2. Zuse'nin Plankalkül

2.1 Tarihsel Arka Plan

Konrad Zuse, 1936–1945 yılları arasında Almanya'da elektromekanik bilgisayarlar inşa etti. 1943'te Z4 bilgisayarı için bir programlama dili geliştirmeye başladı. Bu dili Plankalkül (program hesabı) olarak adlandırdı. Müttefiklerin bombardımanı nedeniyle izole bir köyde çalışan Zuse, 1945'te dili tamamladı ama yayımlamadı. Tanımı 1972'ye kadar yayımlanmadı, bu yüzden çağdaşları üzerinde etkisi olmadı.

2.2 Dilin Özellikleri

Özellik	Açıklama
Temel veri tipi	Tek bit; tamsayı ve kayan nokta bu tipten elde edildi
Kayan nokta	İkilinin tümleyeni + 'hidden bit' şeması (günümüzde de kullanılır)
Yapılar	Diziler (arrays) ve iç içe geçebilen kayıtlar (records)
Döngü	Ada'nın for döngüsüne benzer; goto yok
Seçim	Else cümlesi olmayan if yapısı
Assertion	Java'daki assert'e benzer matematiksel ifadeler

Plankalkül'ün Notasyonu — Örnek Atama

$A[4] + 1$ değerini $A[5]$ 'e ata:

$| A + 1 \Rightarrow A$

$V | 4 \ 5$

$S | 1.n \ 1.n$

İlk satır ifadeyi, V satırı dizi indekslerini, S satırı veri tiplerini gösterir. $1.n = n$ bitlik tamsayı anlamına gelir.

Plankalkül'ün Önemi

Zuse'nin çalışması zamanının çok ötesindeydi. Eğer 1950'lerde bilinmiş olsaydı, veri yapıları ve dil tasarımı çok daha erken gelişecekti. Satranç algoritmaları da dahil çeşitli karmaşık programlar içeriyordu.

3. Pseudocode Sistemleri

1940'ların sonu ve 1950'lerin başında makine kodu yerine programlamayı kolaylaştırmak için çeşitli 'pseudocode' sistemleri geliştirildi. Bu dönemde makine kodu kullanımının iki temel sorunu vardı:

- Talimatlar için sayısal kodlar (ör. ADD = 14) — okunması çok zor
- Mutlak adresleme — herhangi bir ekleme/silme tüm adresleri bozuyor

3.1 Short Code (1949)

John Mauchly tarafından BINAC bilgisayarı için geliştirildi. Matematiksel ifadelerin kodlanmış halleriydi. Yorumlayıcı (interpreter) ile çalıştırıldı — bu yüzden makine koduna göre ~50 kat yavaştı. Derleme yapılmıyordu.

Short Code Örneği

$X0 = \text{SQRT}(\text{ABS}(Y0))$ ifadesi → 00 X0 03 20 06 Y0 şeklinde kodlanırdı. Çarpma operatörü yok — iki operandı yan yana koymak çarpmayı ifade ederdi.

3.2 Speedcoding — IBM 701 (1954)

John Backus tarafından IBM 701 için geliştirildi. 701'i sanal bir üç-adresli kayan noktalı hesap makinesine dönüştürdü. Önemli yenilik: adres yazmaçlarının otomatik artırılması (hardware'de bu özellik ancak 1962'de UNIVAC 1107'de geldi).

Özellik	Değer
Yorumlayıcı yüklenince kalan bellek	700 sözcük
ADD komutu süresi	4.2 ms
Matris çarpımı kodu uzunluğu	12 komut
Programlama hız artışı	~2 haftadan birkaç saate

3.3 UNIVAC 'Compiling' Sistemi — A-0, A-1, A-2 (1951-1953)

Grace Hopper liderliğindeki ekip bu sistemleri geliştirdi. Pseudocode kaynağını makine kodu alt programlarına genişletiyordu (makro genişletme gibi). Gerçek bir derleyici değildi ama önemli bir adımdı.

Cambridge Katkıları

David Wheeler (1950) yeniden konumlandırılabilir adresler geliştirdi. Maurice Wilkes ise alt programları birleştiren ve bellek ayıran assembler tasarladı. Assembly dilleri de bu dönemde gelişti ancak yüksek seviyeli dilleri doğrudan etkilemedi.

4. IBM 704 ve Fortran

1954'te IBM 704'ün duyurulması, kayan noktalı işlemleri donanımda yapmasıyla yorumlayıcı genel kullanımını sonlandırdı. John Backus ve IBM ekibi, ilk yaygın derlenen yüksek seviyeli dil olan Fortran'ı geliştirdi.

4.1 Fortran I (1957)

Kasım 1954'te tasarım raporu yayımlandı ('The IBM Mathematical FORMula TRANslating System: FORTRAN'). Nisan 1957'de derleyici piyasaya çıktı.

Özellik	Fortran I Değeri
Değişken isimleri uzunluğu	6 karaktere kadar
Veri tipi bildirimi	Yok — I,J,K,L,M,N ile başlayan isimler tamsayı, geri kalanlar kayan nokta
Kontrol yapıları	If seçme deyimi, Do döngüsü (704 komutlarına dayalı)
Alt programlar	Var, fakat ayrı derleme yok
G/Ç biçimlendirme	Var
Derleyici geliştirme süresi	18 kişi-yıl; optimizasyon ağırlıklı

İddialı Hedef

Fortran ekibi, üretilen makine kodunun el yazımı kodun yalnızca iki katı yavaş olacağını iddia etti. Çoğu kişi bunu imkânsız gördü. Ama gerçekten başardılar. 1958'de 704 için yazılan kodun yarısı Fortran'la yazılıyordu!

4.2 Fortran Sürümleri — Evrim Tablosu

Sürüm	Yıl	Temel Yenilikler
Fortran II	1958	Bağımsız alt program derleme; hata düzeltmeleri
Fortran IV (66)	1960-62	Açık tip bildirimleri, mantıksal If, alt program parametre geçişi
Fortran 77	1978	Karakter dizisi işleme, mantıksal döngü kontrolü, Else cümlesi
Fortran 90	1992	Dinamik diziler, kayıtlar, pointer'lar, özyineleme, modüller
Fortran 95	1997	Forall döngüsü (paralel programlama), küçük değişiklikler
Fortran 2003	2004	Nesne yönelimli programlama, C ile birlikte çalışabilirlik
Fortran 2008	2010	Co-array'ler (paralel model), DO CONCURRENT
Fortran 2015	2018'de planlandı	Geliştirmeler devam ediyor

4.3 Fortran 95 Örnek Programı

```
! Fortran 95 Örnek
Implicit none
Integer Dimension(99) :: Int_List
Integer :: List_Len, Counter, Sum, Average, Result
Result = 0 ; Sum = 0
Read *, List_Len
If ((List_Len > 0) .AND. (List_Len < 100)) Then
  Do Counter = 1, List_Len
    Read *, Int_List(Counter)
    Sum = Sum + Int_List(Counter)
  End Do
  Average = Sum / List_Len
  Do Counter = 1, List_Len
    If (Int_List(Counter) > Average) Then
      Result = Result + 1
    End If
  End Do
  Print *, 'Values > Average:', Result
End If
```

Fortran'ın Önemi ve Kısıtları

GÜÇLÜ YÖN: İlk yaygın derlenen yüksek seviyeli dil; bilimsel hesaplamayı dönüştürdü.

KISIT: Fortran 90 öncesinde tüm değişken tipleri ve bellek alanları derleme zamanında sabittir — özyineleme imkânsız, dinamik veri yapıları çok zor.

5. Fonksiyonel Programlama: Lisp

Yapay zeka arařtırmalarının ihtiyaçı olan liste işleme, 1950'lerin ortasında yeni bir dil türünü doğurdu. John McCarthy, 1958'de MIT'de Lisp'i tasarladı.

5.1 Lisp'in Gereksinimleri

McCarthy'nin incelediğı cebirsel ifade türevleme probleminden çıkan gereksinimler:

- Özyineleme (recursion) ve koşullu ifadeler — Fortran I'de yok
- Dinamik bağlantılı liste tahsisi
- Kaldırılan (abandoned) listelerin dolaylı serbest bırakılması — garbage collection

5.2 Veri Yapıları: Atomlar ve Listeler

Yapı	Açıklama	Örnek
Atom	Sembol ya da sayısal değer	A, 42, foo
Liste	Parantezler arasında elemanlar	(A B C D)
İç içe liste	Eleman olarak alt liste	(A (B C) D (E (F G)))

5.3 Lisp'in Temel Sözdizimi

Kod ve veri aynı biçimdedir: parantezli listeler. (A B C D) veri olarak okunursa 4 elemanlı listedir; kod olarak okunursa A fonksiyonunun B, C, D parametreleriyle çağrısıdır.

```
; Lisp Örneğı – İki listeyi karşılaştırma
(DEFUN equal_lists (lis1 lis2)
  (COND
    ((ATOM lis1) (EQ lis1 lis2))
    ((ATOM lis2) NIL)
    ((equal_lists (CAR lis1) (CAR lis2))
     (equal_lists (CDR lis1) (CDR lis2)))
    (T NIL)
  )
)
```


5.4 Lisp Türevleri

Dil	Yıl	Özellik
Scheme	1975	Küçük, statik kapsam (static scoping), birinci sınıf fonksiyonlar, eğitim amaçlı
Common Lisp	1980s	Birçok lehçeyi birleştiren büyük standart; hem statik hem dinamik kapsam
ML	1980s	Derleme zamanı tip çıkarımı, Lisp sözdizimi yok, zorunlu programlamayı da destekler
Miranda	1985	Saf fonksiyonel, Haskell'in temeli
Haskell	1992	Saf fonksiyonel, lazy evaluation (tembel değerlendirme)
F#	2010	.NET platformu üzerinde ML/OCaml tabanlı

Lisp'in Mirası

Lisp, 25 yıl yapay zeka uygulamalarına hâkim oldu. Garbage collection, dinamik bellek tahsisi, birinci sınıf fonksiyonlar ve fonksiyonel programlama paradigmasını ilk sunan dildir.

6. İlk Sofistike Adım: ALGOL 60

ALGOL 60, bilimsel uygulamalar için evrensel ve makineden bağımsız bir dil geliştirme çabasının ürünüdür. ACM (Amerika) ve GAMM (Almanya)'dan 4'er üye, Mayıs-Haziran 1958'de Zürih'te buluştu.

6.1 Tasarım Hedefleri

- Sözdizimi standart matematiksel notasyona olabildiğince yakın olsun
- Algoritmalar basılı yayınlarda bu dille betimlenebilsin
- Programlar makine diline mekanik olarak çevrilebilsin

6.2 ALGOL 60'ın Yenilikleri

Yenilik	Açıklama
Blok yapısı (block structure)	begin...end ile yerel kapsam oluşturma; değişken görünürlüğü sınırlandırma
Özyinelemeli alt programlar	Lisp'te zaten vardı; zorunlu diller için ilk kez
İki parametre geçiş biçimi	Pass-by-value ve pass-by-name
Stack-dynamic diziler	Subscript aralığı çalışma zamanında belirlenen diziler
BNF kullanımı	Sözdizimini BNF ile tanımlayan ilk dil; biçimsel dil teorisini başlattı

6.3 ALGOL 60 Örnek Programı

```

comment ALGOL 60 Örneği;
begin
  integer array intlist [1:99];
  integer listlen, counter, sum, average, result;
  sum := 0;
  result := 0;
  readint (listlen);
  if (listlen > 0) ^ (listlen < 100) then
  begin
    for counter := 1 step 1 until listlen do
    begin
      readint (intlist[counter]);
      sum := sum + intlist[counter]
    end;
    average := sum / listlen;
    for counter := 1 step 1 until listlen do
      if intlist[counter] > average
      then result := result + 1;
    printstring('Values > average:');
    printint (result)
  end
  else printstring('Error');
end

```

6.4 ALGOL 60 Neden Yaygınlaşamadı?

Neden	Açıklama
Giriş/Çıkış yok	I/O deyimleri makineden bağımsız yapılamaz diye dışarıda bırakıldı
Pass-by-name	Çok esnek ama anlaşılması ve uygulanması zordu
IBM desteği yok	IBM ve SHARE, Fortran'a yatırımı korumak için ALGOL'u terk etti
BNF yeniliği	1960'ta BNF anlaşılması zor bir notasyon olarak görüldü
Uygulama zorluğu	Tam ALGOL 60 hiçbir derleyicide tam olarak gerçekleştirilmedi

ALGOL 60'ın Mirası

Her zorunlu dil ALGOL 60'a bir şey borçludur: PL/I, SIMULA 67, ALGOL 68, C, Pascal, Ada, C++, Java ve C#. Burroughs bilgisayarları donanım yığıt (stack) mimarisi ile ALGOL'un blok yapısını destekleyecek şekilde tasarlandı.

7. İşletme Uygulamaları: COBOL

Grace Hopper'ın 1953 tarihli önerisi ('iş uygulamaları İngilizce deyimlerle yazılmalı') COBOL'un temelini oluşturdu. İlk resmi toplantı 28-29 Mayıs 1959'da Pentagon'da yapıldı — ALGOL toplantısının tam bir yıl sonrası.

7.1 COBOL'un Temel Özellikleri

Özellik	Açıklama
FLOW-MATIC mirası	İngilizce benzeri sözdizimi FLOW-MATIC'ten alındı
Uzun isimler	30 karaktere kadar, tire ile bağlı anlamlı isimler
Hiyerarşik kayıtlar	Plankalkül'de ortaya çıkmış; COBOL'de ilk uygulandı
Veri bölümü (DATA DIVISION)	Tüm değişkenler burada tanımlanır; ondalık basamak sayısı belirtilir
Prosedür bölümü	Zayıf yön: fonksiyon desteği yoktu, parametrelili alt programlar 1974'e kadar yok
DoD zorunluluğu	ABD Savunma Bakanlığı COBOL kullanımını zorunlu kıldı

COBOL'un Tarihi Önemi

Muhasebenin elektronik mekanizasyonunu mümkün kıldı. 1990'lardaki Y2K araştırmalarında Manhattan'ın 22 km²'sinde düzenli kullanımda yaklaşık 800 milyon satır COBOL kodu olduğu tahmin edildi.

8. Zaman Paylaşımının Başlangıcı: Basic

John Kemeny ve Thomas Kurtz, 1964'te Dartmouth College'da (şimdiki Dartmouth Üniversitesi) liberal sanat öğrencileri için Basic'i geliştirdi. Hedef: bilim/mühendislik dışı öğrencilerin programlama öğrenmesi.

8.1 Basic'in Tasarım Hedefleri

- Bilim dışı öğrencilerin kolayca öğrenebileceği bir dil olsun
- 'Hoş ve dostane' olsun
- Ödevler için hızlı dönüş süresi sağlasın
- Serbest ve özel erişim imkânı tanısin
- Bilgisayar zamanından çok kullanıcı zamanına değer verilsin (devrimci!)

8.2 Temel Özellikler ve Etkisi

Özellik	Açıklama
İlk terminal tabanlı dil	Terminal üzerinden uzak bilgisayara bağlanma imkânı sağlayan ilk yaygın dil
Veri tipi	Yalnızca tek tip: kayan nokta ('sayılar')
Deyim sayısı	Yalnızca 14 farklı deyim türü
Zaman paylaşımı	1964'te aynı anda birden fazla kullanıcı
Visual Basic (1991)	GUI programlamayı basitleştirdi; büyük yeniden doğuş
VB.NET	Nesne yönelimli programlama desteğiyle modernleştirildi

9. Herkes İçin Her Şey: PL/I

IBM, 1963'te bilimsel ve ticari kullanıcıların giderek birbirine yaklaştığını fark etti. Tek bir bilgisayar ailesi (System/360) hem kayan noktalı hem onluk aritmetik yapabilecekti. Buna eşlik edecek tek bir dil — Fortran, COBOL, Lisp ve assembly'nin yerini alacak — tasarlandı: PL/I.

9.1 PL/I'nin İlkleri

- Eş zamanlı (concurrent) çalışan alt programlar oluşturma imkânı
- 23 farklı istisna (exception) türünü algılama ve işleme
- Özyinelemeli alt programlar (devre dışı bırakılabilir)
- Pointer veri tipi
- Dizilerin çapraz kesitlerine (cross-section) erişim — matrisin tek bir satırı gibi

Dijkstra'nın PL/I Eleştirisi (Turing Ödülü Konferansı, 1972)

"Programlama dili — temel aracımız — zaten entelektüel kontrolümüzden çıkmışken, büyüyen programlarımızı kavrayışımız içinde nasıl tutacağımızı anlamakta tamamen başarısız oluyorum."

9.2 PL/I vs ALGOL 68 Karşılaştırması

Kriter	PL/I	ALGOL 68
Yazabilirlik (writability)	Çok sayıda sabit yapı ekleyerek	Az temel + birleştirme mekanizması (ortogonallik)
Hedef alan	Bilim + ticaret + sistem prog.	Yalnızca bilimsel
Başarı	Görece yaygınlaştı (IBM desteği)	Sınırlı kaldı
Öğrenme güçlüğü	Çok büyük dil	Karmaşık metamil (van Wijngaarden)

10. Dinamik Diller: APL ve SNOBOL

Ortak Özellikler

APL ve SNOBOL, dinamik yazım (dynamic typing) ve dinamik bellek tahsisi kullanır. Değişkenler tipten bağımsızdır; bir değer atandığında o değer tipini alırlar. Depolama yalnızca değer atandığında gerçekleşir.

10.1 APL (A Programming Language)

Özellik	Açıklama
Tasarımcı	Kenneth Iverson, IBM, ~1960
Amaç	Başlangıçta bilgisayar mimarisini tanımlamak için
Özellik	Büyük sembol kümesiyle dizilere tek komutlu işlemler (transpoz vb.)
Sorun	Çok sayıda sembol → okunması çok zor → 'tek kullanımlık' programlama dili
Durum	55+ yıldır var; fazla değişmedi; sınırlı kullanım

10.2 SNOBOL

Özellik	Açıklama
Tasarımcılar	Farber, Griswold, Polonsky — Bell Laboratories, 1960'lar
Amaç	Metin işleme (text processing)
Güçlü yanı	String örüntü eşleme (pattern matching) işlemleri
Durum	Hâlâ aktif; metin işleme alanlarında kullanılıyor

11. Veri Soyutlamanın Başlangıcı: SIMULA 67

Norveçli Kristen Nygaard ve Ole-Johan Dahl, 1962-1964 arasında simülasyon uygulamaları için SIMULA I'i geliştirdi. 1967'de SIMULA 67 olarak genişletildi.

11.1 Temel Yenilikler

Kavram	Açıklama
Coroutine	Kaldığı yerden devam edebilen alt programlar — simülasyon için gerekli
Class yapısı	Coroutine desteği için geliştirildi; veri soyutlamanın temeli
Veri soyutlama	Hoare 1972'de sınıf yapısı ile veri soyutlaması arasındaki bağı kurdu
Nesne yönelimli temel	Kalıtım ve dinamik bağlama kavramları buradan geldi

Tarihi Önemi

SIMULA 67 hiçbir zaman yaygın kullanıma ulaşamadı ve döneminin programcıları üzerinde az etki yarattı. Ama tanıttığı sınıf yapısı, nesne yönelimli programlamanın temelini oluşturdu ve Smalltalk, C++ ve Java'ya doğrudan ilham verdi.

12. Ortogonal Tasarım: ALGOL 68

ALGOL 68, ALGOL 60'tan dramatik biçimde farklıdır. Temel tasarım ilkesi ortogonalite'dir: az sayıda temel kavram + sınırsız birleştirme.

12.1 Temel Yenilikler

- Kullanıcı tanımlı veri tipleri — ilk kez
- Flex diziler (implicit heap-dynamic arrays) — ilk kez
- Ortogonalite ilkesinin tasarıma yansması

Neden Yaygınlaşamadı?

Dil, 'van Wijngaarden grameri' adlı bilinmeyen ve çok karmaşık bir metayille tanımlandı. 'Indicants' (anahtar kelimeler), 'trimming' (alt dizi alma), 'coercion of deproceduring' (alt program çağırısı) gibi yabancı terimler öğrenmeyi çok güçleştirdi.

13. Erken ALGOL Torunları: Pascal ve C

13.1 Pascal

Niklaus Wirth, IFIP Çalışma Grubu 2.1'in ALGOL 68'e doğru giden büyük revizyonunu reddettikten sonra kendi ALGOL-W önerisini geliştirdi. 1971'de Pascal'ı yayımladı.

Özellik	Açıklama
Temel yenilik	Öğretim amaçlı; basit+anlamli tasarım
ALGOL-W'den gelen	value-result parametre geçişi, case deyimi
ALGOL 68'den gelen	Kullanıcı tanımlı veri tipleri
Zayıf yön	Değişken uzunluklu dizi parametre geçişi imkânsız; ayrı derleme yok
Popülarite	1970'lerin ortasından 1990'ların sonuna kadar öğretimde birinci dil

```
{Pascal Örneği}
program pases (input, output);
  type intlisttype = array [1..99] of integer;
  var intlist : intlisttype;
      listlen, counter, sum, average, result : integer;
begin
  result := 0; sum := 0;
  readln (listlen);
  if ((listlen > 0) and (listlen < 100)) then
  begin
    for counter := 1 to listlen do
    begin readln(intlist[counter]); sum := sum + intlist[counter] end;
    average := sum / listlen;
    for counter := 1 to listlen do
      if (intlist[counter] > average) then result := result + 1;
    writeln('Values > avg:', result)
    end
  else writeln('Error');
end.
```

13.2 C

C, BCPL → B → C zincirinden gelen ve Dennis Ritchie'nin Bell Laboratories'de 1972'de geliştirdiği sistem programlama dilidir. UNIX'in yazım dilidir.

Özellik	Açıklama
Atalar	CPL, BCPL (Cambridge), B (Thompson, 1970)
Tip sistemi	BCPL ve B tipten bağımsızdı; C tip sistemi getirdi
ALGOL 68 etkisi	for ve switch deyimleri, atama operatörleri, pointer işleme
Standartlar	K&R; C (1978), ANSI C89, C99 (complex, bool, // yorum)
Zayıf yön	Tam tip denetimi yok — esneklik ama güvensizlik
Yayılma nedeni	UNIX'le birlikte bedava derleme ortamı geldi

```
/* C Örneği */
int main() {
    int intlist[99], listlen, counter, sum, average, result;
    sum = 0; result = 0;
    scanf("%d", &listlen);
    if ((listlen > 0) && (listlen < 100)) {
        for (counter = 0; counter < listlen; counter++) {
            scanf("%d", &intlist[counter]);
            sum += intlist[counter];
        }
        average = sum / listlen;
        for (counter = 0; counter < listlen; counter++)
            if (intlist[counter] > average) result++;
        printf("Values > avg: %d\n", result);
    }
    else printf("Error\n");
}
```

14. Mantık Programlama: Prolog

Prolog, bir hesapsal süreci bildirmeye değil, istenen sonucun gerekli biçimini betimlemeye dayanan bir bildirim (declarative / nonprocedural) dildir. Alain Colmerauer ve Phillippe Roussel (Aix-Marseille) ile Robert Kowalski (Edinburgh) tarafından 1970'lerin başında geliştirildi.

14.1 Prolog Veritabanı Yapısı

Yapı	Açıklama	Örnek
Fact (olgular)	Gerçek önermeler	mother(joanne, jake).
Rule (kurallar)	Koşullu çıkarım kuralları	grandparent(X,Z) :- parent(X,Y), parent(Y,Z).
Query (sorgular)	Hedef deyimler	father(bob, darcie).

```
/* Prolog Örneği */
mother(joanne, jake).
father(vern, joanne).

grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

/* Sorgu */
?- grandparent(vern, jake).
/* Cevap: true */
```

Prolog'un Sınırlılıkları

1. Performans: Eşdeğer zorunlu programlara göre çok yavaş
2. Dar uygulama alanı: Yalnızca belirli veritabanı yönetimi ve YZ alanlarında etkili

15. Tarihin En Büyük Tasarım Çabası: Ada

1974'te ABD Savunma Bakanlığı (DoD) 450'den fazla farklı programlama dili kullanıyordu. Gömülü sistemler için hiçbiri uygun değildi. Bu durum Ada'nın doğuşuna yol açtı.

15.1 Tasarım Süreci

Belge	Tarih	Amaç
Strawman	Nisan 1975	İlk gereksinimler
Woodenman	Ağustos 1975	Genişletilmiş gereksinimler
Tinman	Ocak 1976	Daha olgun tanım
Ironman	Ocak 1977	Neredeyse nihai
Steelman	Haziran 1978	Nihai gereksinimler
Ada (MIL-STD 1815)	Temmuz 1980	Standart referans kılavuzu

Neden 1815?

Ada dili, dünyaca ilk programcı sayılan Augusta Ada Byron'un (1815-1851) anısına adlandırıldı. Referans kılavuzundaki standart numarası MIL-STD 1815 da onun doğum yılından geliyor.

15.2 Ada'nın Temel Katkıları

Özellik	Açıklama
Packages (paketler)	Veri soyutlamasını destekler; veri nesneleri + tip tanımları + prosedürler bir arada
Exception handling	Geniş kapsamlı istisna yönetimi
Generic units	Parametreliliği tip/prosedür — yeniden kullanılabilirlik artırır
Concurrency (eş zamanlılık)	Tasks + Rendezvous mekanizması ile görevler arası iletişim

15.3 Ada 95 ve Ada 2005 Yenilikleri

- Ada 95: Kalıtım (inheritance) — classwide types ile polimorfizm; protected objects ile veri paylaşımı
- Ada 2005: Java benzeri interface'ler, zamanlama algoritmaları üzerinde daha fazla kontrol

Ada'nın Günümüzdeki Kullanımı

Ticari ve savunma amaçlı havacılık, hava trafik kontrolü ve demiryolu taşımacılığında hâlâ yaygın biçimde kullanılmaktadır.

16. Nesne Yönelimli Programlama: Smalltalk

Smalltalk, nesne yönelimli programlamayı tam olarak destekleyen ilk dildir. Alan Kay'ın 1960'ların sonunda Utah Üniversitesi'ndeki doktora çalışmasından doğdu; Xerox PARC'ta Smalltalk-72'den Smalltalk-80'e evrildi.

16.1 Temel Kavramlar

Kavram	Açıklama
Mesaj gönderme (message passing)	Tüm hesaplama bir nesneye mesaj gönderilerek yapılır
Her şey nesnedir	Tamsayı sabitlerinden büyük sistemlere kadar her şey nesne
Sınıflar (classes)	SIMULA 67'ye benzer; nesne şablonları
Kalıtım	Tüm sınıflar bir süper sınıftan miras alabilir
Dinamik bağlama	Hangi metodun çalışacağı çalışma zamanında belirlenir

```
"Smalltalk Örneği – Çokgen sınıfı"
class name      Polygon
superclass      Object
instance var     ourPen numSides sideLength

"Çiz"
draw
  numSides timesRepeat: [ourPen go: sideLength;
                        turn: 360 // numSides]

"Kenar sayısını ayarla"
sides: num
  numSides <- num
```

Smalltalk'ın Etkisi

Günümüzde standart haline gelen pencereli arayüz sistemleri (windowing systems) Smalltalk'tan çıkmıştır. SIMULA 67'de doğan nesne yönelimli fikirler Smalltalk'ta olgunluğa ulaştı.

17. C++, Swift ve Delphi

17.1 C++

Bjarne Stroustrup, 1980'de Bell Laboratories'de C'ye SIMULA 67 tarzı sınıfları eklemeye başladı. Adım adım geliştirdi:

Sürüm/Yıl	Yenilikler
C with Classes (1983)	Sınıflar, türetilmiş sınıflar, erişim denetimi, yapıcı/yıkıcı
C++ (1984)	Sanal metotlar (dynamic binding), metot aşırı yükleme, referans tipleri
Release 1.0 (1985)	Cfront: C++ → C çevirisi
Release 2.0 (1989)	Çoklu kalıtım, soyut sınıflar
Release 3.0 (1990)	Şablonlar (templates), istisna yönetimi
ISO C++ (1998)	Standartlaştırıldı
Managed C++ / MC++ (2002)	.NET platformu için

C++'ın Güçlü ve Zayıf Yanları

GÜÇLÜ: C ile geriye dönük uyumluluk; OOP'yi büyük ticari projelerde ilk sunan dil
ZAYIF: C'nin güvensiz özelliklerini miras aldı; PL/I gibi aşırı büyük ve karmaşık

17.2 Swift (2014)

Apple, Objective-C'nin yerine geçmek üzere Swift'i geliştirdi (Chris Lattner, 2010). 2014'te tanıtıldı, 2015'te açık kaynak yapıldı.

- Tip çıkarımı (type inferencing) — tip bildirimi zorunlu değil
- Atama deyimleri değer döndürmez → $x = 0$ boole ifadesinde geçersiz (C hatası ortadan kalkar)
- Referans sayacıyla otomatik bellek yönetimi (ARC)
- Daha güvenli switch: varsayılan olarak sonraki case'e düşmez
- Tuple, optional type, protocol (interface benzeri) yapıları

17.3 Delphi (1995)

Anders Hejlsberg (daha sonra C#'ın tasarımcısı) Pascal'a nesne yönelimli özellikler ekleyerek Delphi'yi oluşturdu. C++'a göre daha zarif ve güvenli ama daha az özellikli.

18. Java

Sun Microsystems, 1990'da gömülü tüketici elektroniği cihazları için yeni bir dile ihtiyaç duydu. James Gosling liderliğindeki ekip, C++ tabanlı ancak daha küçük, basit ve güvenilir Java'yı tasarladı.

18.1 C++ ile Karşılaştırma

Kriter	C++	Java
Pointer	Var (güvensiz)	Yok; referans tipleri var (otomatik dereference)
Çoklu kalıtım	Var	Yok; interface ile bir kısım avantaj sağlanır
Boolean tipi	int olarak kullanılabilir	boolean — aritmetik ifade kullanılamaz
Bellek yönetimi	Manuel (new/delete)	Otomatik çöp toplama (garbage collection)
Ayrık prosedür	Var	Yok — her şey metod; sadece OOP desteklenir
Dizi denetimi	Yok	Var — index range checking
Struct/union	Var	Yok
Eş zamanlılık	Kütüphane bazlı	synchronized — görevler (threads) dil içinde
Portabilite	Derlenmiş makine kodu	JVM + JIT derleme
Tip dönüşümü	Çok geniş coercion	Yalnızca genişletici (widening) coercion

```
// Java Örneği
import java.io.*;
class IntSort {
    public static void main(String args[]) throws IOException {
        DataInputStream in = new DataInputStream(System.in);
        int listlen, counter, sum=0, average, result=0;
        int[] intlist = new int[99];
        listlen = Integer.parseInt(in.readLine());
        if ((listlen > 0) && (listlen < 100)) {
            for (counter=0; counter<listlen; counter++) {
                intlist[counter] = Integer.parseInt(in.readLine());
                sum += intlist[counter];
            }
            average = sum / listlen;
            for (counter=0; counter<listlen; counter++)
                if (intlist[counter] > average) result++;
            System.out.println("Values > avg: " + result);
        }
    }
}
```


Java'nın Yükseliş ve Günümüz Durumu

Başlangıçta tüketici elektroniği için tasarlandı, hiçbir ürün piyasaya çıkmadı. 1993'te Web'in patlamasıyla Java applet'leri çok popüler oldu. JIT derleyiciler sayesinde performans C++'a yakın. Java SE8 (2014) ile lambda ifadeleri, generics, numaralandırma sınıfları eklendi.

19. Betik Dilleri (Scripting Languages)

Betik dilleri, komutların bir dosyaya yazılarak yorumlanmasıyla başladı. sh (shell) ilk örnektir; giderek tam programlama dillerine dönüştüler.

19.1 Perl

Özellik	Açıklama
Tasarımcı	Larry Wall; sh + awk kombinasyonu olarak başladı
Değişken ad uzayları	\$ (scalar), @ (array), % (hash) — okunabilirliği artırır
Örtük değişkenler	\$_, \$!, @_ gibi garip isimler
Diziler	Dinamik uzunluk + seyrek (sparse) diziler + foreach iterasyonu
Hash'ler	String ile indeksli çağrışımsal diziler (associative arrays)
Tehlike	Sayı→string ve string→sayı dönüşümü sessizce hatalara yol açabilir
Kullanım alanı	UNIX yönetimi, Web CGI (artık nadir), biyoinformatik, YZ

19.2 JavaScript

Özellik	Açıklama
Tasarımcı	Brendan Eich, Netscape; Mocha → LiveScript → JavaScript
Standart	ECMA-262; Microsoft sürümü: JScript .NET
Java ile ilişki	Sadece sözdizimsel benzerlik; tip sistemi tamamen farklı
Dinamik yazım	Java statik yazılıdır; JS dinamik yazılıdır
Kullanım	HTML formları doğrulama, dinamik HTML oluşturma, DOM manipülasyonu
OOP desteği	Kalıtım yok; dinamik metot bağlama yok (başlangıç versiyonlarında)

19.3 PHP

Özellik	Açıklama
Tasarımcı	Rasmus Lerdorf, 1994; Apache Group; açık kaynak
Amaç	Sunucu taraflı HTML-gömülü betik — Web uygulamaları
Çalışma şekli	PHP kodu sunucuda işlenir; tarayıcı sadece HTML görür
Özellik	HTML form verilerine kolay erişim; çok veritabanı desteği
Versiyon	PHP 7 (2015'te yayımlandı)

19.4 Python

Özellik	Açıklama
Tasarımcı	Guido van Rossum, Hollanda, 1990'lar; açık kaynak
Yazım	Dinamik yazım; tip denetimi var ama çalışma zamanında
Veri yapıları	Liste, tuple (değişmez liste), dictionary (hash)
OOP	Tam nesne yönelimli + fonksiyonel özellikler
Genişletilebilirlik	C/C++ ile yazılmış modüllerle genişletilebilir
Liste kavraması	Haskell'den ilham; [x*2 for x in liste] sözdizimi
Diğer	Thread, socket, çöp toplama, pattern matching (Perl'den)

19.5 Ruby

Özellik	Açıklama
Tasarımcı	Yukihiro Matsumoto (Matz), Japonya, 1990'lar; 1996'da yayımlandı
Temel özellik	Saf nesne yönelimli — Smalltalk gibi; her şey nesne
Operatörler	Aslında metot çağrısı — yeniden tanımlanabilir
Dinamizm	Sınıf ve nesnelere çalışma zamanında metot eklenebilir
Kapsam	@ = instance değişkeni, \$ = global, küçük harf = yerel
Kültürel önemi	ABD'de yaygın kullanılan ilk Japon programlama dili

20. C# — .NET'in Amiral Gemi Dili

Microsoft, 2000'de .NET platformuyla birlikte C#'ı duyurdu. Tasarımcısı Anders Hejlsberg, Turbo Pascal ve Delphi'nin de tasarımcısıdır.

20.1 C# ile Java/C++ Karşılaştırması

Özellik	Java	C++	C#
Enum tipi	Var (güvenli)	Var (int'e dönüşür)	Var — daha güvenli (int'e örtük dönüşüm yok)
Struct	Yok	Var (az fayda)	Var — gerçek değer tipi
Delegate	Interface ile	Metot pointer	Tip güvenli alt program referansı
Çoklu kalıtım	Yok	Var	Yok (.NET sınırlaması)
Boxing/Unboxing	Java 5'ten itibaren	Yok	Örtük; 2 tip sistemi arasında köprü
foreach	for-each	Yok (std::)	Var
Property	Yok	Yok	Var — get/set otomatik çağrı
Rectangular array	Yok	Yok	Var
Ara temsil	JVM byte kodu	Makine kodu	IL → JIT ile makine kodu

C# 7.0 Yenilikleri

Tuple tipleri ve örüntü eşleme (pattern matching) desteği eklendi. C# sürümleri: 1.0 (2002) → 2.0 → 3.0 → 4.0 → 5.0 → 7.0'a kadar sürekli gelişti.

21. Biçimlendirme-Programlama Melezi: XSLT ve JSP

21.1 XSLT

Kavram	Açıklama
XML	Metamarkup dili — diğer biçimlendirme dillerini tanımlar
XSLT	XML belgelerini HTML/XML'e dönüştüren dil; W3C, 1990'ların sonu
Giriş	XML veri belgesi + XSLT belgesi
Çıkış	Dönüştürülmüş XML/HTML
Template	Veri örüntüsü eşleşince çalışan dönüşüm kuralları (alt program gibi)
Kontrol yapıları	, sıralama (sort) işlemleri

21.2 JSP (Java Server Pages) ve JSTL

Kavram	Açıklama
Servlet	Web sunucusunda çalışan Java sınıfı örneği
JSP	HTML + Java karışımı; JSP işlemcisi bunu servlet'e dönüştürür
JSTL if	Boolean koşuluna göre içerik dahil etme/dışarıda bırakma
JSTL choose/when/otherwise	Çoklu seçim yapısı
JSTL forEach	Koleksiyonlar üzerinde yinleme; begin/end/step öznitelikleri
Kullanım	Form doğrulama, veritabanı erişimi, dinamik Web içeriği

22. Olası Sınav Soruları ve Cevapları

Aşağıdaki açık uçlu sorular, bu bölümden sınavda çıkabilecek türde sorulardır.

S1. Plankalkül hiçbir zaman uygulanmadı ve 1972'ye kadar yayımlanmadı. Buna rağmen neden tarihsel açıdan önemlidir?

Plankalkül, 1945 gibi çok erken bir tarihte tamsayı/kayan nokta/dizi/kayıt veri tiplerini, döngü yapısını (goto olmadan), koşullu seçimi, özyinelemeli olmayan alt program benzeri yapıları ve iddia (assertion) mekanizmasını içeriyordu. Eğer 1950'lerde bilinmiş olsaydı, veri yapıları ve dil tasarımı çok daha hızlı gelişecekti. 'Hidden bit' kayan nokta şeması günümüzde hâlâ kullanılmaktadır.

S2. ALGOL 60, pek çok teknik yenilik getirmesine rağmen Amerika'da hiçbir zaman yaygın kullanıma ulaşamadı. Bunun başlıca üç nedenini açıklayınız.

1) GİRİŞ/ÇIKIŞ EKSİKLİĞİ: I/O deyimleri makineden bağımsız yapılamaz diye dışarıda bırakıldı; bu da taşınabilirliği engelledi. 2) IBM VE SHARE DESTEĞİ YOK: IBM ve SHARE, Fortran'a yaptıkları yatırımı korumak için ALGOL 60'ı desteklemedi. 3) PASS-BY-NAME: Bu çok esnek parametre geçiş mekanizması hem anlaşılması hem de uygulanması güçtü. Hiçbir derleyici tam ALGOL 60'ı gerçekleylemedi.

S3. Lisp'in 'saf fonksiyonel' (pure functional) olmasından ne anlıyoruz? Bu ne tür avantajlar ve dezavantajlar doğurur?

Saf fonksiyonel: Hesaplama tamamen fonksiyon uygulamasıyla yapılır; atama deyimi ve değişkenler zorunlu değildir; yineleme (iteration) özyinelemeyle sağlanır. AVANTAJ: Yan etkisiz programlama → hata ayıklama ve doğrulama kolaylaşır; paralel çalışmaya uygundur. DEZAVANTAJ: Zorunlu (imperative) yaklaşıma alışkın programcılar için öğrenmesi güçtür; bazı problemler özyinelemeyle ifade edildiğinde daha fazla hesaplama gücü gerektirir.

S4. C++, C'nin tüm güvensiz özelliklerini korumaktadır. Stroustrup bu kararı neden aldı?

Stroustrup'un üç temel hedefinden biri 'C++'ın C'nin kullanılabildiği her uygulama için kullanılabilmesi'ydi. Bu yüzden C'nin özellikleri kaldırılmadı. İkinci temel hedef 'C'ye kıyasla performans cezasının olmaması'ydı; dizi indeks denetimi gibi özellikler bu nedenle eklenmedi. Bu yaklaşım C programcılarının C++'a geçişini kolaylaştırdı ve geniş bir kullanıcı kitlesinin hızla oluşmasını sağladı.

S5. Java ve C++ arasındaki en önemli güvenilirlik farklarını karşılaştırınız.

Java'nın C++'a göre güvenilirlik avantajları: (1) Pointer yok — adres aritmetiği ve sarkan işaretçi hataları ortadan kalkar. (2) Otomatik çöp toplama — bellek sızıntısı engellenir. (3) Dizi indeks denetimi — sınır dışı erişim çalışma zamanında yakalanır. (4) Yalnızca genişletici tip dönüşümleri — beklenmedik veri kaybı azalır. (5) Boolean için ayrı tip — C++'daki if (x = 0) hatası derleme hatasına dönüşür.

S6. Ada için neden 'tarihin en büyük tasarım çabası' deniyor? Ada'nın gömülü sistemler için uygunluğunu sağlayan özellikleri nelerdir?

TASARIMIN BÜYÜKLÜĞÜ: DoD'nin 5 aşamalı gereksinim belgesi süreci, 4 finalist arasında rekabetçi seçim, 100'den fazla kuruluşun katıldığı kamu değerlendirme konferansı, 15 ülkeden 500'den fazla dil raporu ile onlarca milyon dolarlık bütçe. GÖMÜLÜ SİSTEM ÖZELLİKLERİ: Exception handling (güvenilirlik); tasks + rendezvous (eş zamanlılık); paketler (büyük sistem organizasyonu); generic birimler (yeniden kullanılabilirlik).

S7. Python, Ruby, JavaScript ve PHP'yi 'betik dili' yapan ortak özellikler nelerdir?

1) Dinamik yazım — değişkenler tip bildirimi gerektirmez. 2) Dinamik uzunluklu diziler ve koleksiyonlar. 3) Çalışma zamanında yorumlama ya da dinamik derleme. 4) Yüksek düzeyli yerleşik veri yapıları (hash, liste vb.). 5) Kısa ve hızlı geliştirme döngüsü. NOT: Bu özellikler esnekliği artırır ama genellikle derleme zamanı tip denetimini ortadan kaldırır ve bazı hataların ancak çalışma zamanında yakalanmasına yol açar.

S8. SIMULA 67 hiçbir zaman yaygın kullanıma ulaşmadı. Neden tarihsel açıdan son derece önemlidir?

SIMULA 67, coroutine desteği sağlamak amacıyla class (sınıf) yapısını tanıttı. Bu yapı, veri soyutlamasının temelini oluşturdu. Hoare 1972'de bu bağı resmi olarak kurdu. Sınıf kavramı daha sonra Smalltalk, C++, Java ve C#'ı doğrudan etkiledi. Nesne yönelimli programlamanın tüm modern mirasının kökü SIMULA 67'dedir.

23. Kitap Bölüm Sonu Soruları ve Çözümleri

Aşağıda kitabın Review Questions ve Problem Set bölümlerinden seçilmiş sorular ve ayrıntılı cevapları verilmektedir.

S1. Plankalkül hangi yılda tasarlandı? Tanımı hangi yılda yayımlandı?

1945 yılında tasarlandı; tanımı 1972 yılında yayımlandı.

S2. Plankalkül'de yer alan iki temel veri yapısı nelerdir?

Diziler (arrays) ve iç içe geçebilir kayıtlar (records, C tabanlı dillerdeki struct'lara karşılık gelir).

S3. 1950'lerin başında pseudocode'lar nasıl uygulamaya konuldu?

Saf yorumlama (pure interpretation) yöntemiyle uygulandı; kaynak kod çalışma zamanında adım adım yorumlandı.

S4. Speedcoding, dönemin bilgisayar donanımının hangi iki önemli eksikliğini gidermeye çalışıyordu?

1) Kayan noktalı (floating-point) donanım desteğinin olmaması — Speedcoding yazılımda kayan noktalı işlemleri sağladı. 2) Dizilerin kullanımını kolaylaştıracak indeksleme (indexing) donanımının olmaması.

S5. 1950'lerin başında yorumlama yavaşlığı neden kabul edilebilirdi?

Kayan noktalı işlemler zaten yazılımda simüle edildiğinden çok zaman alıyordu. Bu ek yük nedeniyle yorumlama maliyeti görece önemsiz kalıyordu.

S6. IBM 704 bilgisayarının hangi donanım özelliği dil evrimini güçlü biçimde etkiledi?

Donanımda kayan noktalı işlem desteği ve indeksleme kayıtları (index registers). Bu özellikler, yorumlama maliyetinin gizlendiği ortamı ortadan kaldırdı ve yüksek seviyeli dillerin derlenmesini zorunlu kıldı.

S7. Fortran I'e eklenerek Fortran II elde edilen en önemli özellik neydi?

Alt programların bağımsız derlenmesi (independent compilation of subroutines). Bu özellik olmadan her değişiklik tüm programın yeniden derlenmesini gerektiriyordu; ayrıca makine güvensizliği nedeniyle pratik program uzunluğu 300–400 satırla sınırlıydı.

S8. Fortran IV'ten Fortran 77 elde edilirken hangi kontrol akışı deyimleri eklendi?

1) Karakter dizisi işleme (character string handling). 2) Mantıksal döngü denetimi deyimleri. 3) İsteğe bağlı Else cümlesi içeren If yapısı.

S9. Hangi Fortran sürümü ilk dinamik değişkeni destekledi?

Fortran 90 — dinamik diziler (dynamic arrays) bu sürümde eklendi.

S10. Lisp hangi iki profesyonel kuruluşun ortak çabası sonucu değil, tek bir kişinin eseridir. Lisp nerede ve kim tarafından geliştirildi?

MIT'de (Massachusetts Institute of Technology), John McCarthy tarafından 1958–1959 yıllarında geliştirildi.

S11. ALGOL 60'ı kimin tasarladı? Hangi iki önemli örgüt bu süreçte yer aldı?

ACM (Association for Computing Machinery — ABD) ve GAMM (Almanya'lı matematik ve mekanik topluluğu) her biri 4 üye göndererek ortak tasarım yaptı.

S12. ALGOL'un hangi sürümünde blok yapısı (block structure) ortaya çıktı?

ALGOL 60.

S13. ALGOL 60'ın yaygınlaşma şansını zedeleyen, neden eksik bırakılan dil öğesi neydi?

Giriş/Çıkış (I/O) deyimleri. Makineden bağımsız yapılamayacağı düşüncesiyle dışarıda bırakıldı; bu da programların farklı bilgisayarlara taşınmasını güçleştirdi.

S14. COBOL hangi programlama dilinden ilham aldı?

FLOW-MATIC — Grace Hopper'ın UNIVAC için geliştirdiği dil; İngilizce'ye benzer sözdiziminin iş uygulamaları için kullanılabileceğini kanıtladı.

S15. COBOL tasarım süreci hangi yılda başladı?

1959 yılında, Savunma Bakanlığı sponsorluğunda Pentagon'daki toplantıyla başladı.

S16. COBOL'da ortaya çıkan ve Plankalkül'de kökü olan veri yapısı nedir?

Hiyerarşik kayıtlar (hierarchical data structures / records).

S17. COBOL'un erken başarısında hangi örgüt en belirleyici rol oynadı?

ABD Savunma Bakanlığı (DoD) — COBOL kullanımını zorunlu kılan ilk örgüt oldu; bu zorunluluk olmasaydı erken derleyicilerin kötü performansı dilin benimsenmesini engellerdi.

S18. PL/I'nin yerini almak üzere tasarlandığı iki dil hangileriydi?

Fortran (bilimsel uygulamalar) ve COBOL (ticari uygulamalar). Bunlara ek olarak LISP ve sistem programlama için kullanılan assembly dili de hedefleniyordu.

S19. PL/I hangi yeni bilgisayar ailesi için tasarlandı?

IBM System/360 bilgisayar ailesi — hem kayan noktalı hem onluk aritmetik yapabilen ilk IBM serisi.

S20. SIMULA 67'nin günümüz nesne yönelimli dilleri için önemli olan özellikleri nelerdir?

Sınıf (class) yapısı ve coroutine desteği. Sınıflar, nesne yönelimli programlamanın temel bileşeni olan veri soyutlamasının başlangıç noktasıdır.

S21. Mantıksal olmayan (nonprocedural) bir dil nedir?

Bir sonucun nasıl hesaplanacağını değil, sonucun ne olması gerektiğini ya da nasıl bir forma sahip olması gerektiğini betimleyen dil. Prolog bu kategorinin en önemli örneğidir; hesaba çözüm arama (resolution) mekanizması yapar.

S22. Prolog veritabanını dolduran iki deyim türü nelerdir?

1) Olgular (facts): Doğrudan doğru olan önermeler (ör. mother(joanne, jake).). 2) Kurallar (rules): Koşullu çıkarım deyimleri (ör. grandparent(X,Z) :- parent(X,Y), parent(Y,Z).).

S23. Ada'nın tasarlandığı birincil uygulama alanı neydi?

Gömülü sistemler (embedded systems) — donanımın kontrol ettiği ya da hizmet verdiği cihazlara gömülü yazılım. Örn. uçak sistemleri, radar, füze güdümlü sistemleri.

S24. Ada'nın eş zamanlı program birimleri ne olarak adlandırılır?

Görevler (tasks). Görevler rendezvous mekanizması aracılığıyla birbirleriyle iletişim kurar.

S25. Ada'da veri soyutlamasını destekleyen yapı nedir?

Paketler (packages) — veri nesnelerini, tip tanımlarını ve prosedürleri kapsülleyen yapılar.

S26. Ada ile COBOL'un ortak yönü nedir?

Her ikisi de ABD Savunma Bakanlığı (DoD) tarafından kullanımı zorunlu kılınan tek dillerdir.

S27. Java'nın ilk uygulaması neydi?

Gömülü tüketici elektroniği cihazları (toaster, microwave oven, interactive TV). Ne var ki bu ürünlerin hiçbiri pazara çıkmadı. Java Web üzerinden popülerlik kazandı.

S28. C++ neden C'nin güvensiz olduğu bilinen özelliklerini içermektedir?

Stroustrup'un C ile Classes için koyduğu hedef: 'C'nin kullanılabildiği her uygulama için kullanılabilmek ve bunu yaparken C'ye kıyasla performans cezasına uğramamak.' Bu nedenle C'nin özellikleri kaldırılmadı.

S29. PHP ile JavaScript'in ilişkisi kullanım açısından nasıldır?

İkisi de Web programlamada kullanılır ama komplementer rollere sahiptir: JavaScript istemci tarafı (tarayıcıda çalışır, form doğrulama, DOM manipülasyonu), PHP ise sunucu tarafı çalışır (veritabanı erişimi, dinamik HTML üretimi).

S30. Python diziler yerine hangi veri yapısını kullanır?

Üç tür liste benzeri yapı: 1) Listeler (lists) — değiştirilebilir. 2) Tuple'lar — değiştirilemez (immutable) listeler. 3) Dictionary (sözlükler) — string ile indeksli hash tabloları. Python 2.0'dan itibaren list comprehension da desteklenmektedir.

Kronolojik Özet — Dil Ailesi Ağacı

Dönem	Dil(ler)	Temel Katkı
1945	Plankalkül	İlk yüksek düzeyli dil kavramı; veri yapıları, döngüler
1949-53	Short Code, Speedcoding, A-0/A-1/A-2	İlk yorumlayıcılar ve ilkel derleyiciler
1957-58	Fortran I-II	İlk yaygın derlenen dil; bilimsel hesaplama
1958-60	ALGOL 58/60	Blok yapısı, BNF, özyineleme; tüm zorunlu dillerin atası
1958-59	Lisp	Fonksiyonel programlama; garbage collection; YZ
1959-60	COBOL	İş uygulamaları; hiyerarşik kayıtlar; uzun isimler
1964	Basic	Terminal tabanlı ilk dil; zaman paylaşımı
1964-67	PL/I	Her şey bir arada; exception handling; concurrency
1960/1962-67	APL/SNOBOL	Dinamik yazım; dizi işlemleri; metin işleme
1962-67	SIMULA 67	Sınıf yapısı; nesne yönelimli programlamanın kökü
1969	ALGOL 68	Ortogonalite; kullanıcı tanımlı tipler; flex diziler
1971/1972	Pascal/C	Öğretim dili (Pascal); sistem programlama (C)
1972	Prolog	Mantık programlama; bildirimsel paradigma
1983	Ada	Gömülü sistemler; packages; generics; concurrency
1980	Smalltalk	Saf OOP; GUI; mesaj tabanlı hesaplama
1985	C++	Zorunlu + OOP; şablonlar; çoklu kalıtım
1990	Python	Betik + OOP; okunabilirlik; genişletilebilirlik
1990/96	Haskell/Ruby	Saf fonksiyonel (Haskell); saf OOP (Ruby)
1993-95	Java/JavaScript	Platform bağımsızlık (JVM); Web istemci betik
1994	PHP	Web sunucu taraflı betik; veritabanı entegrasyonu
2000	C#	.NET bileşen programlama; delegate; property
2014	Swift	Objective-C yerine; güvenli; tip çıkarımı

Son Not

Bu konu anlatımı, Sebasta'nın 'Concepts of Programming Languages' 12. baskısının 2. bölümünü kapsamaktadır. Bölüm içeriğinin tamamı Türkçe olarak aktarılmış; teknik terimler İngilizce orijinallerini koruyacak şekilde parantez içinde gösterilmiştir.